University of Science and Technology of Oran – Mohamed Boudiaf
Faculty of Mathematics and Informatics
Department of Computer Science

TP 2 : Tries and Bloom Filters

Phone numbers: prefixes and spam calls

Mohammed Said BELAID
said.belaid@univ-usto.dz

10/11/2026

Objectives

- Implement a trie and use it to search by prefix.
- Implement a Bloom filter and count its false positives.

In this TP, a phone number is a string of 10 digits, such as `"0555123456"`. Use only **randomly generated** numbers, never real ones.

Exercise 1: A phone book trie

In this exercise, we implement a **trie** (prefix tree) that stores phone numbers. The alphabet has only 10 symbols: each node has `children[10]` (the child reached by digit `c` is `children[c - '0']`) and a boolean `isEnd`, true when a number ends at this node.

1. Implement the trie: the node structure, `insert(number)` and `contains(number)`. Give their complexity as a function of the length L of the number.
2. Write `startsWith(prefix)`, which returns all the numbers that start with `prefix`.

```
insert("0555123456"); insert("0555987654"); insert("0661000111")
startsWith("0555")  ->  ["0555123456", "0555987654"]
startsWith("07")    ->  []
```

3. Write `randomNumber()`, which returns a random 10-digit number starting with `05`, `06` or `07`. Insert $n = 10^5$ random numbers into the trie. Does the complexity of `contains` depend on n?
4. Count the nodes of the trie for $n = 10^3$, 10^4 and 10^5, and compare with the $10n$ digits stored one after the other.

Exercise 2: Blocking spam calls

A phone keeps a list of known spam numbers. The list holds millions of numbers, too many to store exactly on the phone, so we use a Bloom filter: an array of m bits, all 0 at the start. A 10-digit number fits in a 64-bit integer, so our hash functions are simple operations on the number x.

1. **One hash function.** Take $h_1(x) = x \bmod m$. Implement `add(x)` (set bit $h_1(x)$ to 1) and `mightContain(x)` (is bit $h_1(x)$ equal to 1?). With $m = 10^6$, which digits of the number does h_1 use?



2. Insert $n = 10^5$ random spam numbers. Check that `mightContain` answers "maybe" for every one of them. Why can the filter never answer "no" for a spam number? Then test 10^5 random numbers that are **not** spam (check it with a hash set) and count the false positives.

3. **A second hash function.** Add $h_2(x) = x \bmod 999983$ (a prime number close to m). Unlike h_1, it depends on **all** the digits of x. Now `add` sets **both** bits, and `mightContain` checks both. With the same m, count the false positives again. Is it better?